

BiblioGen API REST

Modern Library Access Interface

v1.0 | RESTful API | JSON | Python/Django

Disciplina: Construção de APIs para Inteligência Artificial (UFG)

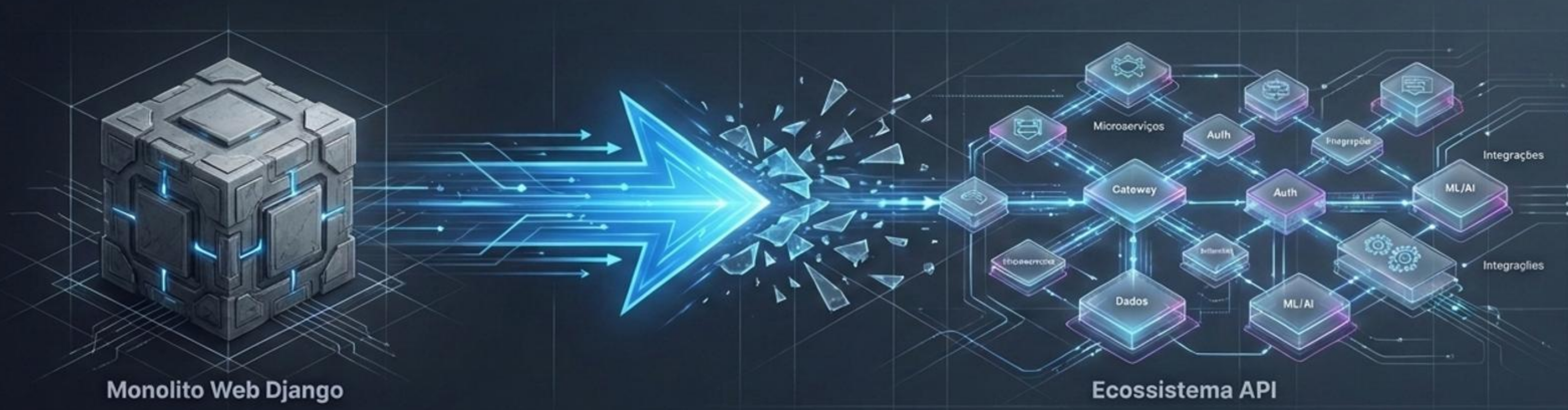
Professor: Rogério

Equipe: Antonio, Jucelino, Vanderson, Ronny, Givanildo.

Arquitetura da Solução: O Motor Principal



Os Quatro Pilares da API BiblioGen



Pilar 1: Segurança

Proteção de rotas com SimpleJWT e controle de permissões em múltiplas camadas.



Pilar 2: Inteligência Cognitiva

Integração de Busca Vetorial Local e LLM na Nuvem (RAG).



Pilar 3: Integridade & Rastreabilidade

Validações estritas de dados e log de auditoria RequisicaoIaLog.



Pilar 4: Garantia de Qualidade

13 testes automatizados de integração, OpenAPI (Swagger) e Postman automatizado.



Pilar 1: Controle de Acesso e Segurança JWT

POST /api/v1/auth/token/

Keys and Locks

👁 **Perfil:** Leitor (Acesso Comum)

📋 **Permissão:** Somente Leitura

→ **Métodos:** GET

📋 **Escopo:** Consulta de acervo e uso do assistente de IA.

cryptographaiaic.token

🔑 **Perfil:** Bibliotecário (Admin)

✎ **Permissão:** Escrita e Modificação

→ **Métodos:** POST, PUT, DELETE

📁 **Escopo:** Gerenciamento de livros (/api/v1/livros/), usuários e empréstimos.

Mecânica Base: Emissão de Access e Refresh Tokens via SimpleJWT no DRF.

Pilar 2: Inteligência Cognitiva Dual

Recomendação Semântica (Execução Local)

POST /api/v1/ia/recomendar/

Motor: Sentence-Transformers
(paraphrase-multilingual-MiniLM-L12-v2)

Técnica: Geração de vetores de 384 dimensões e similaridade de cosseno direto na CPU.

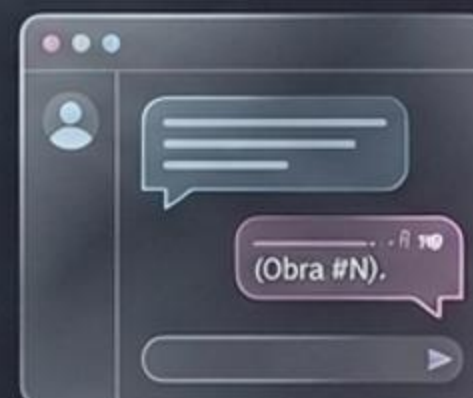


Assistente Conversacional RAG (Execução Cloud)

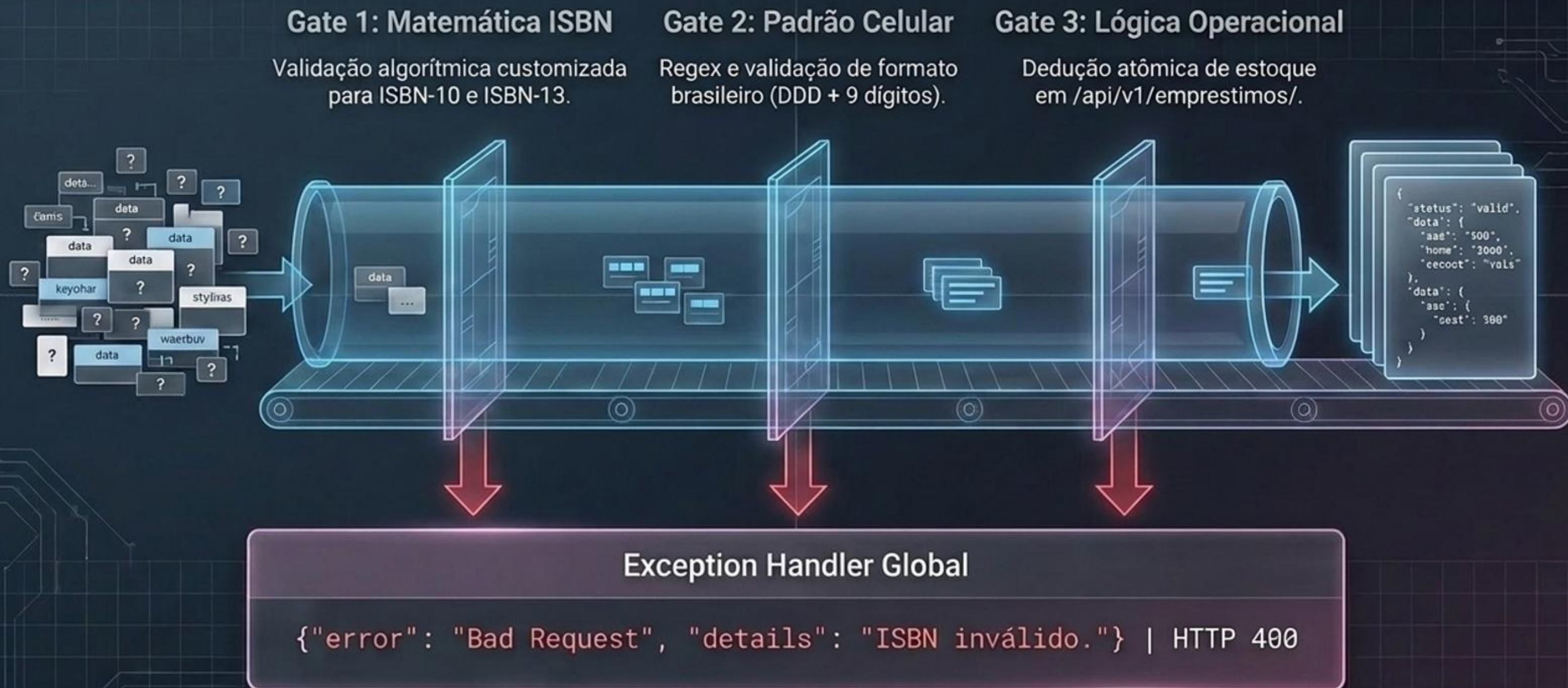
POST /api/v1/ia/chat/

Motor: Groq Cloud API
(llama-3.3-70b-versatile)

Técnica: Recuperação híbrida com latência ultrabaixa, citando fontes no formato (Obra #N).



Pilar 3: Integridade Estrita de Dados

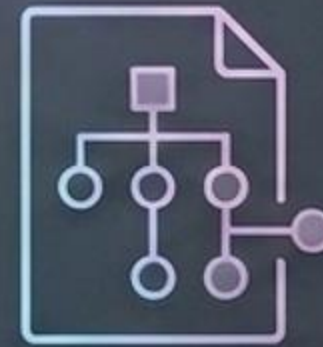


Pilar 4: Garantia de Qualidade (QA)

13

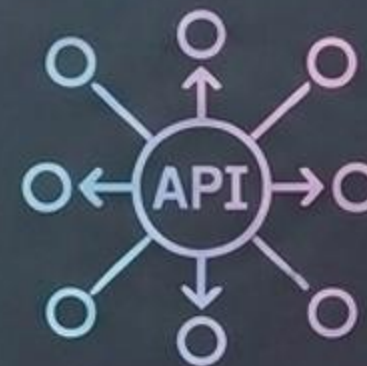
Testes de Integração Automatizados (APITestCase)

Cobre login JWT, permissões de grupo, limites no chat de IA (<3 ou >1000 letras) e fluxo transacional de empréstimo.



OpenAPI 3.0

Documentação viva e interativa gerada dinamicamente via drf-spectacular acessível em `/api/v1/docs/`.



Postman Collections

Coleção JSON com script de ambiente pós-requisição que captura e injeta o `access_token` silenciosamente nas rotas.

Auditoria e Rastreabilidade de IA

O que a inteligência processa, o sistema registra.

Tabela: RequisicaoIaLog

ID	Usuário Solicitante	Pergunta (Input)	Obras Citadas (FK)	Timestamp
104	admin_biblio	"Livros de banco de dados?"	[12, 15]	2026-06-25 14:32:01



Loggers Rotativos: Registros em console e django.log para monitoramento infraestrutural.

Demonstração Funcional: O Cliente SPA

A materialização do backend em uma interface interativa.

Gestão de Sessão:

Login JWT integrado com armazenamento seguro no localStorage do navegador.

Consumo Direto:

Listagem e cadastro de obras consumindo os endpoints DRF nativamente via JS Puro.



Interação RAG:

Painel de chat integrado renderizando cards dinâmicos das obras recomendadas em tempo real.



Servido via `/api/v1/dashboard-cliente/`

Entrega de Valor: Matriz de Requisitos

[✓]	Requisitos Básicos Atendidos Tratamento de erros JSON, validação de dados matemática e versionamento de rotas v1.
[✓]	Múltiplos Serviços de IA Recomendação semântica e Assistente Chatbot RAG implementados de forma independente.
[✓]	Execução Remota & QA 13 testes de integração, containerização pronta e garantia de execução agnóstica.
[✓]	Maturidade Técnica (Entregas Extras) Implementação de logs de auditoria de IA e injeção automatizada de JWT.

```
git clone https://github.com/jucelinoss/biblioteca-django-rag
```